

Caso di studio – UNO: il gioco di carte

DOCUMENTAZIONE PROGETTUALE	1
INTRODUZIONE E ARCHITETTURA DEL SISTEMA.....	1
COMPONENTI	1
RISPETTO DEI REQUISITI TECNICO-IMPLEMENTATIVI.....	1
<i>Strutture dati e tipi (dati strutturati)</i>	1
Entità di Dominio (Record fondamentali)	1
Entità di flusso e stato.....	2
Nodi per le strutture dati dinamiche	2
<i>Strutture dati dinamiche</i>	2
<i>Modularizzazione (ADT e AD)</i>	3
<i>Acquisizione, input e file management</i>	3
FLUSSO OPERATIVO E INTERFACCIA UTENTE.....	3
SUGGERIMENTI PER OTTIMIZZARE IL FLUSSO	4

Documentazione progettuale

Introduzione e architettura del sistema

Il presente documento illustra l'analisi, la progettazione e l'implementazione di una trasposizione digitale del celebre gioco di carte "UNO". Il software è sviluppato in linguaggio **C**, utilizzando l'IDE **Code::Blocks**.

L'architettura del sistema è progettata per essere modulare, scalabile e manutenibile. L'interfaccia grafica (GUI) e la gestione degli input (mouse/tastiera) sono affidate alla libreria **Raylib**, che garantisce un'interazione fluida e reattiva. L'applicativo supporta sia una modalità in giocatore singolo (contro avversari controllati dall'IA) sia una modalità multigiocatore in rete locale o remota tramite l'utilizzo di **Socket**.

Componenti

Alessio Decarolis - 852865

Pasquale Franco - 852866

Rispetto dei requisiti tecnico-implementativi

Il progetto è stato ingegnerizzato per soddisfare rigorosamente i vincoli didattici richiesti, mappando ogni struttura dati e tecnica di programmazione a una specifica meccanica di gioco.

Strutture dati e tipi (dati strutturati)

- **struct (Typedef):** Utilizzate estensivamente per definire i record fondamentali del dominio applicativo. Esempi includono la struttura Utente (Nome, Cognome, Username, Email, Password), la struttura Carta (Colore, Valore, Tipo) e la struttura Statistiche (Partite giocate, Modalità, Piazzamento, Tempo).

Entità di Dominio (Record fondamentali)

- Gestione carta

```
typedef struct {
    char colore[20]; // es. "Rosso", "Verde", "Blu", "Giallo", "Nero"
    int valore;      // 0-9. Per carte speciali si può usare un codice es. -1
    char tipo[20];   // es. "Normale", "CambioGiro", "Salta", "Piu2", "Jolly", "JollyPiu4"
} Carta;
```

- Gestione statistiche

```
typedef struct {
    int partite_giocate;
    char modalita[30]; // es. "Singolo", "Multiplayer"
    int piazzamento; // es. 1 (Vittoria), 2, 3, 4
    float tempo;      // Durata della partita in secondi/minuti
} Statistiche;
```

- Gestione utente

```
typedef struct {
    char nome[50];
    char cognome[50];
    char username[30];
    char email[100]; // Funge da chiave primaria per il database
    char password[50]; // Eventualmente cifrata
    Statistiche stats; // Statistiche associate all'utente
} Utente;
```

Entità di flusso e stato

- Gestione giocatore

```
typedef struct {
    Utente info;          // Dati dell'utente o nome del Bot
    int is_bot;           // 1 se è controllato dall'IA, 0 se è reale
    int stato_pronto;     // 1 per "Pronto", 0 per "In attesa" (nella Lobby)
    int num_carte_mano;   // Serve a fine partita per stilare la classifica
} Giocatore;
```

- Gestione interazioni multiplayer

```
typedef struct {
    char mittente[30];    // Username di chi invia il messaggio
    char testo[256];      // Contenuto del messaggio
    char orario[10];      // Timestamp dell'invio (es. "14:35")
} MessaggioChat;
```

Nodi per le strutture dati dinamiche

- Gestione nodo per la mano del giocatore (lista doppiamente concatenata)

```
typedef struct NodoMano {
    Carta carta;
    struct NodoMano* next;
    struct NodoMano* prev;
} NodoMano;
```

- Gestione nodo per il mazzo di desca e degli scarti (Pila / Stack)

```
typedef struct NodoPila {
    Carta carta;
    struct NodoPila* next;
} NodoPila;
```

- Gestione nodo per i turni di gioco (coda/queue)

```
typedef struct NodoCoda {
    Giocatore giocatore;
    struct NodoCoda* next;
    struct NodoCoda* prev;
} NodoCoda;
```

- Gestione nodo per gli utenti in RAM (albero binario di ricerca - BST)

```
typedef struct NodoAlbero {
    Utente utente;
    struct NodoAlbero* left;
    struct NodoAlbero* right;
} NodoAlbero;
```

Strutture dati dinamiche

- **Array e Matrici:** Utilizzati per gestire il rendering a schermo degli elementi grafici tramite Raylib (es. una matrice di coordinate per il posizionamento delle carte sul tavolo o dei bottoni nei menu).
- **Lista (Lista Concatenata):** Implementata per gestire la **mano di ciascun giocatore**. Poiché il numero di carte in mano varia continuamente (pescando o scartando), una lista doppiamente concatenata permette inserimenti ed eliminazioni efficienti senza sprecare memoria.
- **Pila (Stack):** Utilizzata per il **Mazzo di Pesca** e il **Mazzo degli Scarti** (logica LIFO - *Last In, First Out*). Le carte giocate vengono "pushate" sulla pila degli scarti, e il colore/numero valido per il turno successivo è sempre quello in cima (top()) alla pila.
- **Coda (Queue):** Fondamentale per la gestione dei **Turni di Gioco** (logica circolare). I giocatori (reali o bot) sono inseriti in una coda. Al termine di un turno, il giocatore viene rimosso dalla testa e inserito in coda. La carta speciale "Cambio Giro" semplicemente inverte la logica di scorrimento della coda (o utilizza una coda bidirezionale/deque).
- **Albero (Tree):** Implementazione di un **Albero Binario di Ricerca (BST)** per caricare in memoria RAM gli utenti al momento del Log In o per gestire le Statistiche. Questo garantisce una ricerca logaritmica rapida dell'Email (chiave primaria) in fase di autenticazione, evitando la scansione sequenziale del file ad ogni tentativo di accesso. In alternativa, un Albero decisionale verrà utilizzato per strutturare la logica dell'IA dei bot.

Modularizzazione (ADT e AD)

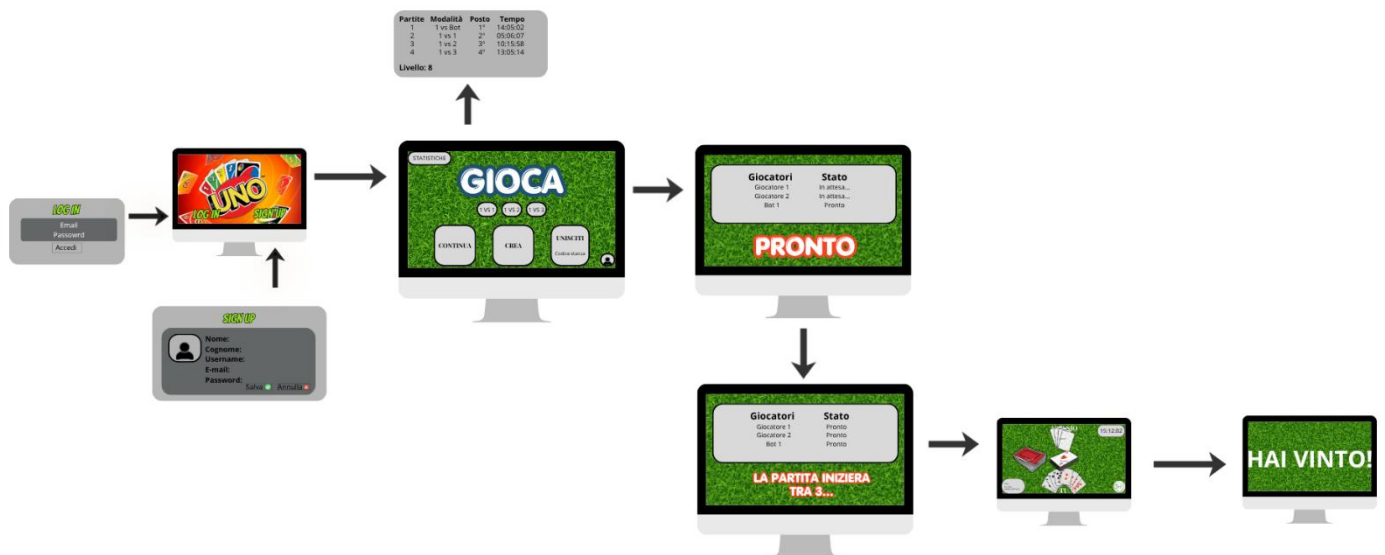
Il codice seguirà il paradigma degli *Abstract Data Types* (ADT). Le strutture dati complesse (Pila, Code, Liste) e le loro operazioni di manipolazione saranno isolate in moduli separati (.h e .c dedicati), garantendo l'information hiding. Il progetto sarà diviso in moduli logici:

- auth.h/.c: Gestione registrazione e accesso.
- game_logic.h/.c: Regole di UNO, validazione mosse.
- network.h/.c: Gestione socket (Host/Client).
- ui.h/.c: Funzioni di rendering Raylib.

Acquisizione, input e file management

- **Acquisizione da Tastiera/Mouse:** Gestita interamente dai listener di Raylib.
- **File Sequenziale:** Utilizzato per il salvataggio o l'esportazione di log testuali (.txt) o per le statistiche di fine partita.
- **File ad Accesso Diretto (Random Access):** Utilizzato per il database degli utenti (.dat). Grazie alla funzione fseek(), sarà possibile aggiornare direttamente il record di uno specifico utente (ad esempio, per aggiornare le statistiche a fine partita) senza dover riscrivere l'intero file. L'Email funge da chiave primaria per il controllo dei duplicati.

Flusso operativo e interfaccia utente



Il ciclo di vita dell'applicazione, come da schema grafico, è strutturato in fasi sequenziali:

1. Autenticazione (Schermata Iniziale):

- **Sign Up:** Creazione nuovo profilo. I dati vengono salvati sul file utenti dopo un check sulla chiave primaria (Email).
- **Log In:** Inserimento credenziali. Il sistema verifica la corrispondenza nel file e sblocca l'accesso.

2. Dashboard Principale:

- **Profilo e Statistiche:** Lettura da file per mostrare lo storico delle partite (Modalità, Posto, Tempo, Livello).
- **Selezione Modalità:** Creazione partita singola (1vs1, 1vs2, 1vs3 contro Bot) o multigiocatore (Crea Stanza / Unisciti tramite codice). In caso di giocatori umani insufficienti, il sistema istanzia automaticamente i bot mancanti.

3. Lobby (Stanza di Attesa):

- Riepilogo dei giocatori connessi o dei bot.
- Gestione degli stati ("In attesa..." / "Pronto"). I bot assumono automaticamente lo stato Pronto.

4. Gameplay Loop (Tavolo Verde):

- Un timer globale in C cronometra la durata del match.
- I giocatori utilizzano il mouse per selezionare la carta dalla propria mano (Lista) e giocarla sul banco (Pila).
- Le validazioni sono effettuate in tempo reale dal modulo game_logic.
- È possibile interagire con gli altri utenti in partita tramite una chat.

5. Termine Partita (Schermata "Hai vinto!/Hai perso!"):

- Condizione di vittoria: La lista della mano di un giocatore punta a NULL (0 carte).
- Il sistema ordina i perdenti in base alla lunghezza (numero di nodi) della loro lista della mano.

- Salvataggio dei dati di fine match (durata, piazzamento) nel file .dat delle statistiche.
- Reindirizzamento alla Dashboard o Log Out.

Suggerimenti per ottimizzare il flusso

In fase di sviluppo verranno implementate ulteriori funzioni quali:

- **Il tasto "UNO":** A livello di UI, verrà inserito un pulsante a schermo o un tasto rapido da premere quando un giocatore rimane con una sola carta. Se un giocatore reale non lo preme prima che il turno passi, e un altro utente lo segnala, dovrà pescare 2 carte (puoi gestire questo con un timer asincrono o un evento listener di Raylib).
- **Sicurezza dei dati (File):** Visto che le password verranno salvate in un file .dat/.txt, verrà implementata una funzione di cifratura (anche un semplice cifrario di Cesare o un'operazione XOR).
- **Disconnessioni nelle Socket:** Se un giocatore in rete si disconnette improvvisamente durante la partita, il gioco potrebbe bloccarsi aspettando il suo turno all'infinito. Implementa un controllo sulle socket: se la connessione cade, quel giocatore viene automaticamente sostituito da un Bot per permettere agli altri di finire la partita.
- **Gestione della Pila del Mazzo:** In UNO, quando il mazzo di pesca finisce, si prendono gli scarti (tranne l'ultima carta in cima), si mescolano e diventano il nuovo mazzo di pesca. A livello strutturale, verrà implementata una funzione che riversi gli elementi della Pila Scarti nella Pila Mazzo e applichi un algoritmo di shuffling (es. *Fisher-Yates*).